

Same Model, Different Words: How Prompt Phrasing Changes What Claude Produces

Tiger Zhang September 2026

Abstract

The same language model gives a generic answer to one phrasing of a request and a usable one to another. This paper asks which phrasing choices matter, by how much, and whether the answer depends on the task or the model. We review the published literature and vendor guidance, then run a controlled experiment: twelve tasks across six domains (coding, writing, analysis, extraction, reasoning, and agentic planning), each written in ten prompt patterns that hold the underlying task fixed and vary only the wording, each run as a fresh subagent on three Claude models (Fable 5.1, Opus 5, Sonnet 5), for 360 runs. Outputs were scored by a blind judge on correctness, completeness, adherence, usefulness, and concision, plus objective checks where the task allowed them. Across 360 runs, the full-stack prompt (role, goal, context, constraints, format) scored 9.12 out of 10 against 5.46 for a bare command, and the ordering of patterns was the same on all three models. Format and constraints were the strongest single additions; a role or a chain-of-thought cue added almost nothing on its own; context helped only where the task depended on facts the model could not infer. Sonnet 5 with a full prompt outscored Fable 5.1 with a bare one, so wording moved output quality more than model choice did. A survey of the prompts people share, across GitHub, vendor guides, Reddit and Hacker News, and widely viewed posts, found that the most-used one is the interview, the pattern the experiment ranked second; the most common line in shared prompts, a role opener, added nothing in the experiment. The findings are packaged as a Claude Code skill that collects project context, grades a draft prompt out of 100, interviews the user in one batch, and produces three rewrites with the patterns named, and as a website with the pattern library, results, and a live grader. The skill covers code, agent, and writing prompts, and was itself put through the same blind-graded procedure over five rounds, which raised its mean rewrite score from 92.6 to 96.1 and exposed one regression its own self-assessment had missed.

1. Introduction

Prompt engineering is programming in natural language. The instruction "write something about our product" and the instruction "you are a senior B2B copywriter; write a two-sentence LinkedIn ad for ops managers at mid-sized companies, confident but not salesy, ending with a call to action" go to the same model, and the difference in the result comes entirely from the words. Most people write the first kind. Two things

make that costly now. Models are used for actions, not only text: they edit repositories, call tools, and send messages, so a vague prompt produces a vague action. And the models have changed under the advice: instructions that fixed lazy behaviour in 2024 now cause over-triggering, and the chain-of-thought cue that lifted arithmetic scores in 2022 mostly adds latency on models that reason on their own.

This project has three parts. The research part collects what is known, from 52 papers and the current documentation of Anthropic, OpenAI, and Google, adds a controlled experiment on the Claude family, and checks the result against the prompts people actually share. The product part is a Claude Code skill, `/prompt-engineer`, that turns the findings into a repeatable workflow: read the project, grade the draft, ask the few questions that change the rewrite, rewrite three ways, and explain which patterns were used. The website presents the pattern library, the experiment results with a blind rating page, and a grader that runs the rubric in the browser.

We are not proposing a new technique. The contribution is a ranking, with evidence, of the techniques people already use, measured on the models people use today, plus a tool that applies the ranking to the prompt in front of you using facts from the project it will run in. That last step, adapting the generic advice to one codebase or one document, is where most of the gain comes from. It is also where most guides stop, because a guide cannot open your repository.

2. What is known

2.1 The hierarchy: goal, role, context, action, format

Practitioner guides converge on the same skeleton. The Prompting Guide decomposes a prompt into instruction, context, input data, and output indicator. Google's whitepaper and the two teaching videos that motivated this project describe a hierarchy of goal, role, context, action, and output format. Anthropic's documentation says the same in a different order and adds a test: show the prompt to a colleague with no context, and if they would be confused, so is the model.

The empirical support for specificity is strong. Yang et al. (2025) found that models infer unstated requirements only 41% of the time and that under-specified prompts are twice as likely to regress when the model or the prompt changes. Bsharat et al. (2023) reported quality gains of roughly 50% across 26 principles, but a Wharton replication (Meinke et al. 2025) showed that the structural principles (state the audience, delimit the task, decompose steps, let the model ask) hold while the incentive principles (tips, threats, emotional appeals) do not replicate on 2025 models.

2.2 Role prompting

The role is the most-recommended and least-supported element. Zheng et al. (2023) tested 162 personas across four model families on 2,410 factual questions and found no accuracy gain over a no-persona control. A December 2025 Wharton report on six frontier models found no expert persona that consistently improved GPQA and nine

significant degradations on MMLU-Pro. Where role prompting does help (Kong et al. 2023), the mechanism looks like an accidental reasoning trigger. The practical reading is that a role sets voice and taste, not knowledge; it belongs in writing tasks and not in factual ones.

2.3 Context and the why

Anthropic's documentation asks for the motivation behind each rule ("this is read aloud by text-to-speech, so no ellipses") on the grounds that Claude generalises from the reason. The limit on context is placement and relevance. Liu et al. (2023) showed a U-shaped attention curve: material in the middle of a long prompt is used least. Gloaguen et al. (2026) evaluated repository context files for coding agents and found that generic overviews do not lift success and raise cost by over 20%, while non-standard rules the agent could not infer do help. Context earns its place when it is a fact the model cannot get elsewhere.

2.4 Constraints and negatives

Explicit constraints improve adherence, but with two limits. IFEval (Zhou et al. 2023) showed that even GPT-4 missed roughly one prompt in four when constraints were verifiable, so stacking many constraints in one prompt lowers the chance any one is met. Negation is parsed unreliably (Truong et al. 2023), and Rana (2026) showed that naming a forbidden word primes it: 87.5% of violations in 40,000 samples were priming failures. Anthropic's guidance is to say what to do instead of what not to do, and to drop "CRITICAL" and "ALWAYS" because current models over-trigger on them. Our reading is that a constraint should name the failure mode it prevents ("do not ask them to retry; they already tried twice"), which gives the model a reason and a positive alternative at once.

2.5 Format and examples

Format is a large, under-controlled variable. Sclar et al. (2023) found accuracy spreads of up to 76 points from formatting alone; He et al. (2024) found up to 40% swings on smaller models between plain text, Markdown, JSON, and YAML. Few-shot examples teach format and label space more than content (Min et al. 2022), which is why two or three examples in the wanted voice are the most reliable way to control style. Examples hurt on RL-trained reasoning models (DeepSeek-R1 report) and on some large code models (Cheng and Mastropaolo 2026), and example order is a hidden hyperparameter (Lu et al. 2022).

2.6 Reasoning cues

"Let's think step by step" moved GSM8K from 10% to 41% on InstructGPT (Kojima et al. 2022). On models that reason internally the picture reversed. Sprague et al. (2024) found chain-of-thought gains concentrate on math and symbolic tasks; the Wharton Report 2 (Meincke et al. 2025) found reasoning models gain about 3% for 20 to 80% more latency and one model lost 13 points; Liu et al. (2024) found losses of up to 36 points on tasks where deliberation hurts humans too. Anthropic's current advice is to

prefer "think thoroughly" over a hand-written step plan and to control depth with the effort parameter.

2.7 Asking before acting

The interview pattern, where the model asks clarifying questions before producing anything, has the strongest recent evidence for agents. ClarifyGPT (Mu et al. 2023) raised GPT-4 pass@1 by roughly ten points on code generation with one round of questions. Ambig-SWE (Vijayvargiya et al. 2025) found interaction improves performance by up to 74% on under-specified software tasks, and that the hard part is detecting ambiguity, not using the answer. Anthropic's own prompts gate the behaviour with a decision test: check in only when different readings would lead to materially different work.

2.8 Machine-written prompts

APE (Zhou et al. 2022), OPRO (Yang et al. 2023), Promptbreeder, DSPy, and TextGrad all show that when a metric exists, a model searching over wordings beats a human writing them. OPRO's "Take a deep breath and work on this problem step-by-step" scored 80% on GSM8K against 72% for the human phrase. The Prompt Report's case study found a DSPy-optimised prompt produced in minutes beat twenty hours of expert iteration. The implication for a prompt-improvement tool is that the tool should propose several rewrites and let the user or a check pick, rather than assert one.

2.9 How Anthropic phrases its own instructions

We read Claude Code's skills and Anthropic's published system prompts as a case study and catalogued 24 phrasing patterns. The recurring ones: describe the why, not just the rule; state the failure mode the rule prevents; attach behaviour to a recognisable trigger ("when X, do Y"); state the degree of freedom; give a default with an escape hatch; define done and demand evidence; name anti-patterns specifically; ration emphasis to one line; and assume the model is smart and cut what it already knows. These are the patterns the skill applies when it rewrites.

3. Method

3.1 Tasks

Twelve tasks, two per domain:

Domain	Task	Ground truth
coding	code-1 parse ISO-8601 durations	10 hidden test cases
coding	code-2 find two bugs, ignore a red herring	expected findings
writing	write-1 support reply to a failed export	rubric with five signals and word limit
writing	write-2 LinkedIn post on a 4-day week	fact sheet, length, banned openers

Domain	Task	Ground truth
analysis	analysis-1 meeting transcript to decisions and actions	5 decisions, 6 owners
analysis	analysis-2 compare three fictional tools	invented facts and pricing
extraction	extract-1 feedback lines to ticket titles	expected area labels, 60 chars
extraction	extract-2 messy receipts to JSON	exact expected array
reasoning	reason-1 discount, tax, and change	one number
reasoning	reason-2 five-slot scheduling puzzle	unique assignment
agentic	agent-1 plan a feature in a described codebase	right files, migration, tests, a risk
agentic	agent-2 scope a vague product brief	facts, non-goals, milestones, risks

Each task has a hidden rubric the model under test never sees, and shared material (a complaint, a transcript, receipts) that is identical across variants.

3.2 Patterns

Ten variants per task. P0 is a bare command. P1 is a precise task statement and is the base for P2 to P7, which each add exactly one element: role, context, constraints with negatives, output format, few-shot examples, or a chain-of-thought cue. P8 combines role, goal, context, constraints, and format in the hierarchy order. P9 tells the model to ask clarifying questions and supplies the answers in the same prompt, simulating an interview.

3.3 Models and isolation

Each variant was run as a fresh Claude Code subagent on Fable 5.1, Opus 5, and Sonnet 5 with the variant text as its entire input. Subagents ran from an empty directory. A first batch was discarded when we found that subagents launched from the project folder could open the hidden rubrics; a contamination check on every saved output scans for rubric-only strings, and any hit is re-run.

3.4 Scoring

A separate judge agent receives, per task, the hidden rubric and all outputs with model and pattern labels removed and order shuffled, and scores each output from 0 to 10 on correctness, completeness, adherence, usefulness, and concision, plus pass or fail on the rubric's objective checks. The judge is told not to reward length. Human ratings are collected on the website's blind rating page and reported separately when available.

3.5 Limitations

One judge model scores outputs from its own family. One run per cell, so per-cell numbers carry noise and only pattern-level and domain-level aggregates should be read. The tasks are short; long-context effects are not measured. The interview pattern is simulated with pre-supplied answers, which measures the value of the answers rather than the model's ability to ask well.

4. Results

Table 1. Mean blind-judge score by prompt pattern (0 to 10, average of five dimensions; n = runs per pattern; objective pass = share of rubric checks passed).

Pattern	n	Total	Correct	Complete	Adherence	Useful	Concise	Objective pass
P0 bare	36	5.46	6.50	6.08	4.42	5.03	5.25	57%
P1 specific	36	7.09	7.92	7.69	6.33	6.78	6.75	73%
P2 role	36	6.94	7.83	7.97	5.97	6.61	6.31	69%
P3 context	36	7.13	8.25	8.08	5.86	6.83	6.61	73%
P4 constraints	36	7.88	8.25	8.33	7.50	7.58	7.75	83%
P5 format	36	8.40	8.61	8.67	8.25	8.03	8.44	94%
P6 few-shot	36	8.12	8.50	8.50	7.78	7.75	8.06	87%
P7 chain-of-thought	36	6.99	8.53	8.33	6.14	6.89	5.06	74%
P8 full stack	36	9.12	9.33	9.39	9.14	9.08	8.64	99%
P9 interview	36	8.75	9.28	9.31	8.47	8.69	8.00	95%

Table 2. Pattern by domain (mean total; each cell averages two tasks and three models).

Pattern	agent	analysis	code	extract	reason	write
P0 bare	5.13	6.10	6.67	1.80	8.30	4.73
P1 specific	6.87	6.60	5.47	9.13	8.60	5.90
P2 role	7.20	6.17	5.37	8.60	8.87	5.43
P3 context	6.37	6.70	5.80	7.70	8.73	7.47
P4 constraints	7.33	8.63	7.40	9.13	9.10	5.70
P5 format	8.10	8.13	9.17	9.43	8.50	7.07
P6 few-shot	8.40	8.13	7.43	8.93	9.40	6.40
P7 chain-of-thought	7.73	6.50	7.13	6.40	9.03	5.13
P8 full stack	8.60	9.13	9.37	9.53	9.43	8.63
P9 interview	7.97	8.77	8.37	9.47	9.27	8.67

Table 3. Pattern by model (mean total across all tasks).

Pattern	Fable 5.1	Opus 5	Sonnet 5
P0 bare	6.07	5.20	5.10
P1 specific	7.75	6.68	6.85
P2 role	7.50	6.80	6.52

Pattern	Fable 5.1	Opus 5	Sonnet 5
P3 context	7.35	7.25	6.78
P4 constraints	8.48	7.48	7.68
P5 format	8.52	8.43	8.25
P6 few-shot	8.17	7.83	8.35
P7 chain-of-thought	7.45	6.83	6.68
P8 full stack	9.18	9.05	9.12
P9 interview	9.00	8.88	8.37

Table 4. Bare versus full stack per task (mean over three models) and the best pattern on that task.

Task	Bare (Po)	Full stack (P8)	Gain	Best pattern
agent-1	4.87	8.80	+3.93	P6 few-shot (8.80)
agent-2	5.40	8.40	+3.00	P9 interview (8.47)
analysis-1	5.40	8.80	+3.40	P8 full stack (8.80)
analysis-2	6.80	9.47	+2.67	P5 format (9.53)
code-1	4.40	8.93	+4.53	P8 full stack (8.93)
code-2	8.93	9.80	+0.87	P5 format (9.93)
extract-1	1.60	9.20	+7.60	P1 specific (9.47)
extract-2	2.00	9.87	+7.87	P5 format (10.00)
reason-1	8.13	9.60	+1.47	P8 full stack (9.60)
reason-2	8.47	9.27	+0.80	P9 interview (9.47)
write-1	1.93	8.60	+6.67	P8 full stack (8.60)
write-2	7.53	8.67	+1.13	P9 interview (8.73)

4.1 The ordering

Table 1 gives the main result. The full stack (P8) scored 9.12, the interview form (P9) 8.75, and a bare command (Po) 5.46. The single additions rank format (8.40), few-shot (8.12), constraints (7.88), then a cluster near the specific statement alone: context (7.13), specific (7.09), chain-of-thought (6.99), role (6.94). The objective checks agree with the judge: the full stack passed 99% of rubric checks, the bare prompt 57%.

The gap between Po and P1 is the largest single step in most domains and is entirely a matter of saying what is wanted. Extraction moves from 1.80 to 9.13 because the bare prompt ("make these into ticket titles") produced explanations, numbering, and dossiers instead of six lines; the specific prompt named the shape. Writing moves from 4.73 to 5.90 and then to 7.47 with context, because the support-reply task needs facts (the cause, the timeline, the sign-off) that no amount of care can invent; the bare and specific prompts either asked questions back or made the facts up, and the judge scored both low.

4.2 What each addition buys

Format (P5) was the best single addition and won three tasks outright, including a perfect 10 on receipt extraction. Its effect shows up on adherence (8.25 against 6.33 for P1) and concision (8.44 against 6.75). On the coding tasks the format instruction did something more basic: it told the model to put the code in the reply. Without it, the Opus and Sonnet subagents often wrote the function to a file and returned a prose summary, which is useless to a reader of the message; with it, every model returned one fenced block. In an agentic setting, where the output goes is part of the format.

Constraints (P4) raised adherence by 1.2 points and concision by 1.0 over P1, and won the analysis domain's summarisation task, where the constraint was a word limit and a ban on invented items. Their cost was visible in writing (5.70): a list of things not to say, without the facts to say instead, produced careful but empty replies. This matches the negation literature: a prohibition without a positive target does not tell the model what to do.

Few-shot (P6) helped most where voice or shape mattered (writing, tickets, the feature plan, where it won) and least where the examples could be copied. Two outputs in the extraction task reproduced an example's wording rather than the input's.

Context (P3) surprised us. It was decisive on the one task built to need it (write-1: 1.93 to 7.40 on its own) and nearly neutral elsewhere, and it lowered concision (6.61) because the models used the extra material to write more. Context is not free. It pays when it carries facts the model would otherwise guess, and it costs when it is background the model already has, because the model treats it as a cue to write more.

Role (P2) scored 6.94, below the specific statement it was added to. It moved tone in the writing tasks and nothing else, in line with the persona studies in Section 2.2. Chain-of-thought (P7) raised correctness slightly (8.53, the highest of the single additions) and cut concision to 5.06, the lowest of any pattern, because the models did what they were asked and showed their working. On the reasoning tasks every run of every pattern reached the right answer, so the cue bought nothing there; it cost most on writing, where reasoning in the reply is noise.

4.3 Interview versus full stack

The interview form (P9) matched the full stack on correctness and completeness (9.28 and 9.31 against 9.33 and 9.39) and trailed on adherence and concision, because several outputs echoed the question-and-answer material or added a preamble. Where the facts were supplied it was as good as writing them into the prompt; on the vague product brief (agent-2) it was the best pattern, because the Q&A format put the missing facts where the model looked for them. This is the simulated version: the model was handed the answers. The value of asking depends on the user being there to answer. Section 6 shows it is also the prompt people share most.

4.4 Across models

The ranking held on all three models (Table 3): P8 first and P9 second on every model, P0 last on every model, and the same middle cluster. The gap changed. Fable 5.1 started higher on the bare prompt (6.07 against 5.20 and 5.10) and gained less from the full stack (+3.12 against +3.85 for Opus and +4.02 for Sonnet). Sonnet 5 with the full-stack prompt (9.12) outscored Fable 5.1 with the bare prompt (6.07) by three points. For a task that will be run many times, a better prompt on a cheaper model is a better deal than a bare prompt on the strongest one.

4.5 Where it did not matter

Both reasoning tasks were solved by every run (36 of 36 correct final answers each), and the coding bug-hunt was found by every run. On tasks the model already gets right, the patterns only moved presentation: whether the answer was on its own line, whether the code was in one block, whether the reply stopped when it was done. That is still the difference between an output someone can paste and one they have to edit. It is also why adherence and concision, not correctness, separate the patterns in Table 1: on tasks like these, everyone gets the answer, and the prompt decides how much cleanup follows.

4.6 Harness effects worth knowing

Two artifacts of running the study inside Claude Code affected the numbers, and we would rather report them than tidy them away. First, on code-1, subagents in the same batch sometimes wrote to the same scratch file, so four Fable runs were re-run one at a time, and several Opus and Sonnet runs returned summaries of a file instead of the code; those scored low on usefulness as delivered, which depresses the P0 to P3 cells in the code column. Second, the judge's Python was 3.9, and three otherwise correct solutions failed to import because of a `float | int` annotation; we re-ran those three with 3.10 semantics and corrected their objective checks. Neither artifact changes the ordering.

5. From findings to a tool

5.1 The skill

/prompt-engineer reads before it judges. In a repository that means CLAUDE.md, the README, manifests, recent commits, and any file the draft names; if the draft names a concept rather than a file, it finds the file that implements it. In a document folder it means the brief, the rubric, prior drafts, the feedback on them, and a sample of the user's own writing. It grades the draft with a seven-dimension rubric out of 100, one line of evidence per dimension. It asks three to six questions in one batch, skipping any the project already answers and saying what it inferred instead. Then it gives three rewrites: minimal (the two highest-value fixes), structured (the full hierarchy with project facts substituted for generic nouns), and interview or agentic. It names the patterns applied and the ones deliberately left out, and ends with what would push the score higher.

The targets are code and prose alike: Claude Code in a repository, chat, an API system prompt, a subagent brief, and writing (essays, articles, letters, reports) either in chat or in a shared document folder. The target decides the shape of the rewrite. A system prompt gets no bracketed placeholders, because the file is loaded verbatim and a literal bracket reaching production is worse than a documented default. A subagent brief gets an absolute path and a return schema, because a subagent has no working directory and no memory of the conversation. A writing rewrite ends with a self-check reported under a separator so the piece itself copies clean.

5.2 Why project substitution is the hard part

Every guide says "add context". The gain comes from the specific context: the real test command, an existing test to imitate, the branch name, the rule in CLAUDE.md. A prompt that says "follow the style guide" is worse than one that says "match tests/test_dates.py". Generic advice cannot supply these; a tool running inside the project can. The skill's context step exists for this reason, and its rubric scores context by whether the facts are project-specific, not by whether a context section exists.

5.3 Testing the tool with the paper's own method

A tool built from a measurement should be measured the same way, so the skill was put through the procedure in Section 3. Eight drafts spanning five targets (a one-line bug fix, a vague feature request, "write docs", an API system prompt, a subagent brief, a scholarship essay, a history paper in a document folder, and a cover letter) were run through the skill, and each recommended rewrite was scored by an independent grader against the skill's own rubric, blind to the skill's self-grade. Five rounds followed, each fixing only what a grader had named and then re-testing.

Round	Mean score	Weakest draft
Baseline	92.6	85
1	94.1	89
2	95.0	90
3 to 5	96.1	92

Three results are worth reporting. First, the fixes that moved scores were not the ones that sounded most principled. Requiring every self-check to produce an inspectable artifact, a word count or a list of sentences over thirty words rather than a claim that the model had reread its work, was worth more than any rewording of the pattern advice. Second, replies got shorter as they got better; the bug-fix reply fell from 96 lines to 61 while its score rose. Third, round 1 caused a regression before it caused an improvement. A new rule placed a bracketed placeholder on the single fact a prompt turned on, which pushed that draft from 95 to 89; rounds 2 and 3 recovered it by requiring the keystone placeholder to sit at the top of the prompt and outside the acceptance criterion. That regression was invisible to the skill's own self-assessment and was caught only because a separate grader scored each rewrite. The lesson

generalises beyond this tool: a prompt-improvement system that grades its own output will not find the cases where its rules fight each other.

5.4 Website

The site carries the pattern library with evidence strength, the results tables and charts, the survey of the prompts people use most (Section 6), a blind side-by-side rating page whose scores are stored and can be merged with the judge scores, a grader that applies the rubric to a pasted prompt, and the paper.

6. What people actually use

The experiment measures what each pattern does to the output. It says nothing about what people actually type. To compare the two, we surveyed the prompts people shared in the year to September 2026 in four places: GitHub prompt and skill collections, the vendors' own guides, Reddit and Hacker News, and widely viewed posts on X, LinkedIn, Substack, and YouTube. A prompt counted as used when it recurred across independent sources, or when a platform reported installs for that one prompt. Repository stars measure a whole collection rather than any prompt in it, so they only broke ties. Every star count, install count, post metric, and Hacker News score below was re-read from its source on 27 and 28 September 2026.

6.1 The most-used prompt

The interview is the most-used prompt. The user describes an idea in a line; the model questions them until every decision is settled, writes the result to a spec file, and a fresh session builds from that file. Its older chat form is one sentence asking the model to keep asking clarifying questions until it is 95% confident it can do the task.

It is the only template in the top five of all three sources that can be ranked, and Anthropic's own guides include it. Packaged as the grill-me skill (Pocock 2026), it is the most-installed prompt on the skills.sh leaderboard, with 1.2 million installs; the one entry above it is a tool for finding other skills. Seven GitHub skill collections carry a version. The post that brought it to Claude Code users (Shihipar 2025) drew 2.33 million views, and Anthropic later added the prompt to its Claude Code best-practices page and its prompt library. On Reddit and Hacker News the 95% form turned up in fourteen places, and interview or Socratic variants in eleven more.

This is P9. In the experiment it scored 8.75, second only to the full stack at 9.12, and it was the best pattern on the vague product brief. So the prompt practitioners converged on is the one our blind judge ranked highest after the full stack. The popular versions also do two things our runs did not test. They ask in rounds until nothing is left open, and they finish by writing a file that a new session executes. The skill's interview step already batched its questions, put a recommended answer on each, and looked facts up instead of asking for them; after the survey, its pattern reference gained the rounds-and-file form for builds too large for one sitting.

6.2 The rest of the top ten

Table 5 orders templates by how many of the four source families carry them, then by the largest reach we could verify. First place is clear on both counts. Below it the order is a judgement call.

Table 5. The ten most-used prompt templates, September 2026 (families: GitHub collections, viral posts, vendor guides, Reddit and Hacker News).

#	Template	Families (of 4)	Largest verified reach
1	Interview first, or ask clarifying questions until 95% confident	4	1.2M installs (Pocock 2026)
2	Give the model a check it can run and ask for evidence	4	8.23M views (Cherny 2026a)
3	Plan before writing code	4	976 points (Hacker News 2026)
4	Simplicity and surgical changes	4	215K stars (Chang 2026)
5	A separate critic loops until the work passes a bar	4	5.05M views (Shumer 2026)
6	Write each correction into CLAUDE.md	4	9.24M views (Cherny 2026b)
7	Brutal honesty, no praise	3	20+ Reddit places
8	Terse output, no filler	3	108K stars (caveman 2026)
9	Write the test first, then make it pass	3	7 GitHub collections
10	An explainer or motion-graphics video from one prompt	2	18.85M views (Remotion 2026)

The audiences differ. On Reddit, where most posters use chat models for everyday work, anti-flattery prompts are the most reposted family, found in more than twenty places. On GitHub and X, where the posters mostly run coding agents, the leading templates are procedures: interview me, verify, plan, loop until a critic passes the work, write the correction down. Most of them are one or two sentences long. The older "act as X" collections still hold large star counts (171K for the best known), but none of their prompts recur anywhere else.

6.3 Where practice and measurement disagree

The most common single line in shared prompts is the role opener, "You are a senior X". It appears in nearly every collection and in fifteen Reddit and Hacker News threads. In our runs, adding a role to an otherwise specific prompt moved the mean from 7.09 to 6.94. It is cheap to write, which may be why it spreads, but on these tasks it did not help.

The full stack, our best pattern, does circulate, though as a fill-in framework rather than a fixed prompt. The most-upvoted post of the year on r/PromptEngineering is one such

framework (context, task, constraints, format), and structured frameworks of this kind turned up in ten places.

Advice from Anthropic's Claude Code team points the other way on examples. In July 2026 Cat Wu and Thariq Shihpar said that examples in a system prompt are no longer best practice for current models, that lists of things not to do lower quality, and that Claude Code's own system prompt had shrunk by 80% (Willison 2026). In our runs, few-shot examples were the second-strongest single addition overall, at 8.12, and in the top three on every model. The runs used task prompts and the advice concerns system prompts, so both can be true. Testing examples inside a system prompt is the obvious next experiment.

6.4 Limits of the survey

Stars and installs are proxies. Only `skills.sh` counts installs for a single prompt, and an install is not a use. Reddit's pages were closed to automated reading and its RSS feeds carry no vote counts, so we re-checked the subreddit ranking only for `r/PromptEngineering`, and other upvote figures are as reported in press coverage. Place counts are lower bounds, and the four searches counted places in slightly different ways. The note behind this section, with every source and caveat, is `research/popular-prompts.md` in the repository.

7. Conclusions

Wording moves output quality more than model choice does, and the wording that works has a recognisable shape. Say what is wanted and for whom; give the facts the model cannot infer and only those; name the constraints and the failure they prevent; specify the format down to where the output goes; and, when the facts are missing, ask before writing. Do that and the cheapest model in the family beats the strongest one given a bare command. Add a role only for voice. Skip the chain-of-thought cue on current Claude models unless a bare answer is being asked for on a multi-step problem. Never stack emphasis. People who share prompts have reached much of this on their own. The most-used prompt online is the interview, though the line they add most often, the role, did the least here.

For anyone using Claude Code, the practical version is shorter. Most of the gain is in substitution, which is tedious and nobody does it in a hurry: the real test command instead of "the tests", an existing file instead of "the style guide", the branch name instead of "recent changes". A prompt cannot contain what its author did not bother to look up. The `/prompt-engineer` skill exists to do the looking up, grade what was written, ask the three questions that would change the answer, and hand back a prompt that carries the project's own facts. The rating page on the site collects the human judgement this paper lacks; the next version should add other model families, longer inputs, and a second judge.

References

- Agarwal, R., et al. (2024). Many-Shot In-Context Learning. NeurIPS 2024. arXiv:2404.11018.
- Anthropic (2026). Prompting best practices; Prompting Claude Fable 5.1; Agent Skills best practices; Claude Code best practices; Claude Code prompt library. platform.claude.com and code.claude.com, read September 2026.
- Anthropic (2024 to 2025). Building effective agents; Writing tools for agents; Effective context engineering for AI agents; Equipping agents for the real world with Agent Skills. anthropic.com/engineering.
- Basil, S., Shapiro, I., Shapiro, D., Mollick, E., Mollick, L., & Meincke, L. (2025). Playing Pretend: Expert Personas Don't Improve Factual Accuracy. Wharton Generative AI Labs.
- Bsharat, S. M., Myrzakhan, A., & Shen, Z. (2023). Principled Instructions Are All You Need for Questioning LLaMA-1/2, GPT-3.5/4. arXiv:2312.16171.
- caveman (2026). A skill for terse output. GitHub, github.com/JuliusBrussee/caveman.
- Chang, F. (2026). andrej-karpathy-skills: a CLAUDE.md derived from Andrej Karpathy's notes on LLM coding. GitHub, github.com/forrestchang/andrej-karpathy-skills.
- Cheng, Z., & Mastropaolo, A. (2026). An Empirical Study on the Effects of System Prompts in Instruction-Tuned Models for Code Generation. arXiv:2602.15228.
- Cherny, B. (2026a). Post on X describing how he uses Claude Code, 2 January 2026. x.com/bcherny/status/2007179832300581177.
- Cherny, B. (2026b). Post on X with tips for using Claude Code, 31 January 2026. x.com/bcherny/status/2017742741636321619.
- DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.
- Dobariya, O., & Kumar, A. (2025). Mind Your Tone: Investigating How Prompt Politeness Affects LLM Accuracy. arXiv:2510.04950.
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2026). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv:2602.11988.
- Hacker News (2026). How I use Claude Code: Separation of planning and execution. news.ycombinator.com/item?id=47106686, 22 February 2026.
- He, J., et al. (2024). Does Prompt Formatting Have Any Impact on LLM Performance? arXiv:2411.10541.
- Khattab, O., et al. (2023). DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. ICLR 2024. arXiv:2310.03714.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large Language Models are Zero-Shot Reasoners. NeurIPS 2022. arXiv:2205.11916.
- Kong, A., et al. (2023). Better Zero-Shot Reasoning with Role-Play Prompting. NAACL 2024. arXiv:2308.07702.

- Li, C., et al. (2023). Large Language Models Understand and Can be Enhanced by Emotional Stimuli. *arXiv:2307.11760*.
- Liu, N. F., et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*. *arXiv:2307.03172*.
- Liu, R., et al. (2024). Mind Your Step (by Step): Chain-of-Thought can Reduce Performance on Tasks where Thinking Makes Humans Worse. *arXiv:2410.21333*.
- Lu, Y., Bartolo, M., Moore, A., Riedel, S., & Stenetorp, P. (2022). Fantastically Ordered Prompts and Where to Find Them. *ACL 2022*. *arXiv:2104.08786*.
- Meincke, L., Mollick, E. R., Mollick, L., & Shapiro, D. (2025a). Prompting Science Report 1: Prompt Engineering is Complicated and Contingent. *arXiv:2503.04818*.
- Meincke, L., Mollick, E. R., Mollick, L., & Shapiro, D. (2025b). Prompting Science Report 2: The Decreasing Value of Chain of Thought in Prompting. *arXiv:2506.07142*.
- Meincke, L., Mollick, E. R., Mollick, L., & Shapiro, D. (2025c). I'll pay you or I'll kill you, but will you care? Wharton Generative AI Labs.
- Min, S., et al. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? *EMNLP 2022*. *arXiv:2202.12837*.
- Mu, F., et al. (2023). ClarifyGPT: Empowering LLM-based Code Generation with Intention Clarification. *arXiv:2310.10996*.
- OpenAI (2025). GPT-4.1 prompting guide; GPT-5 prompting guide; Reasoning best practices. *developers.openai.com*.
- Pocock, M. (2026). skills, including grill-me and grilling. *GitHub*, *github.com/mattpocock/skills*.
- Rana, S. (2026). Semantic Gravity Wells: Why Negative Constraints Backfire. *arXiv:2601.08070*.
- Remotion (2026). Post on X announcing Agent Skills for making videos with Claude Code, 20 January 2026. *x.com/Remotion/status/2013626968386765291*.
- Saravia, E., et al. Prompt Engineering Guide. *promptingguide.ai*, read September 2026.
- Schulhoff, S., et al. (2024). The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. *arXiv:2406.06608*.
- Sclar, M., Choi, Y., Tsvetkov, Y., & Suhr, A. (2023). Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. *ICLR 2024*. *arXiv:2310.11324*.
- Shihpar, T. (2025). Post on X on spec-based development with Claude Code, 28 December 2025. *x.com/trq212/status/2005315275026260309*.
- Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*. *arXiv:2303.11366*.
- Shumer, M. (2026). Posts on X introducing the Gauntlet Loop, 25 to 27 July 2026. *x.com/mattshumer_/status/2081100592689324502*.
- skills.sh (2026). Agent skills leaderboard. *skills.sh*, read 27 September 2026.

- Sprague, Z., et al. (2024). To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. ICLR 2025. arXiv:2409.12183.
- Suzgun, M., & Kalai, A. T. (2024). Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding. arXiv:2401.12954.
- Truong, T. H., Baldwin, T., Verspoor, K., & Cohn, T. (2023). Language models are not naysayers. *SEM 2023. arXiv:2306.08189.
- Vijayvargiya, S., Zhou, X., Yerukola, A., Sap, M., & Neubig, G. (2025). Ambig-SWE: Interactive Agents to Overcome Underspecificity in Software Engineering. ICLR 2026. arXiv:2502.13069.
- Wang, X., et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. arXiv:2203.11171.
- Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903.
- Willison, S. (2026). A Fireside Chat with Cat and Thariq from the Claude Code team. simonwillison.net, 21 July 2026.
- Yang, C., et al. (2023). Large Language Models as Optimizers. ICLR 2024. arXiv:2309.03409.
- Yang, C., Shi, Y., Ma, Q., Liu, M. X., Kästner, C., & Wu, T. (2025). What Prompts Don't Say: Understanding and Managing Underspecification in LLM Prompts. arXiv:2505.13360.
- Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.
- Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023. arXiv:2305.10601.
- Yin, Z., et al. (2024). Should We Respect LLMs? A Cross-Lingual Study on the Influence of Prompt Politeness on LLM Performance. SICon 2024. arXiv:2402.14531.
- Zheng, M., Pei, J., Logeswaran, L., Lee, M., & Jurgens, D. (2023). When "A Helpful Assistant" Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. Findings of EMNLP 2024. arXiv:2311.10054.
- Zhang, M. J. Q., & Choi, E. (2023). Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs. arXiv:2311.09469.
- Zhou, J., et al. (2023). Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911.
- Zhou, Y., et al. (2022). Large Language Models Are Human-Level Prompt Engineers. ICLR 2023. arXiv:2211.01910.
- Full notes with quotations and URLs are in the repository under research/.